

Comprehensive 3D Multi-Organ CT Segmentation Report

End-to-End Quantitative and Qualitative Analysis

MedSegFM Project Workspace

April 19, 2026

Abstract

This detailed report documents the end-to-end medical segmentation pipeline built for the AMOS22 dataset in the MedSegFM workspace. It details exact quantitative outcomes, dataset statistics, failure categorization percentages, and extensive qualitative visualization mappings. The objective was to create a strict fairness-enforced experimentation system to evaluate pretraining, adaptation, and prompting strategies for building a unified hybrid segmentation model. This report captures the methodology, numerical evaluations, and visual diagnostics used to construct the final artifact.

1 Project Timeline and Build Path

The project was not completed in a single pass. It moved through a sequence of linked stages, where each stage created the inputs needed by the next one and surfaced the engineering issues that had to be resolved before the final hybrid model could be trusted.

Stage	Why it was done	What it produced
Raw dataset audit	To distinguish the true AMOS data from archive residue and confirm which folders could be used safely.	Verified that <code>amos22/</code> was the real dataset and <code>__MACOSX/</code> was only metadata residue.
Step 2 assembly and cleaning	To pair images and labels, detect missing cases, and create deterministic splits that every later experiment could reuse.	600 discovered cases, 360 usable cases, 240 missing pairs, and split manifests for train/val/test.
Step 3 pre-processing	To normalize orientation, spacing, and intensity so the model saw consistent 3D volumes.	Preprocessed NIfTI volumes, manifest metadata, and a visual sanity-check figure.
Baseline training stack	To establish a reference model and a reproducible training loop.	CLI commands, checkpointing, validation logs, inference, and qualitative outputs.
Fairness locks and variant comparisons	To ensure that component comparisons were actually apples-to-apples.	Fairness fingerprints, run summaries, and filtered comparison tables.
Failure analysis and final hybrid build	To understand failure modes and then assemble the best final configuration from the available evidence.	Final-hybrid config, comparison tables, packaged inference scripts, and the final report/PDF.

The important point is that the final model was not chosen by intuition. It was assembled from a chain of reproducible artifacts: manifests, configs, training histories, validation outputs, fairness checks, comparison CSVs, and failure-review plots.

Contents

1	Project Timeline and Build Path	1
2	Introduction and Clinical Motivation	3
3	Dataset Demographics and Quality Assurance	3
3.1	Data Materialization Statistics	3
3.2	Standardized Preprocessing (Step 3)	3
4	Training Loop, Validation Flow, and Smoke Runs	4
4.1	How training works	4
4.2	Why this mattered	4
5	Model Infrastructure and Fairness Standardization	5
5.1	Baseline Architecture	5
5.2	Fairness Enforcement	5
6	Module Map and Responsibilities	5
7	Final Hybrid Synthesis and Quantitative Results	6
7.1	How the selector works	6
7.2	What the archived selection actually chose	7
7.3	Why this was the final configuration	7
7.4	Final comparison numbers	8
7.5	Why the later reruns were not used for fair comparison	8
8	Observed Quantitative Outcomes and Organ-Level Results	8
8.1	Per-organ ranking from the smoke test	8
8.2	Failure taxonomy used by the evaluator	9
9	Comprehensive Visual Failure Analysis and Diagnostics	9
9.1	Full Evaluation Profile	9
9.2	Divergence: Strong vs Weak Output Analysis	10
9.3	Extreme Failure Mode Categorization	10
10	Engineering Issues Encountered and Resolved	11
11	Conclusion and Deliverables	11

2 Introduction and Clinical Motivation

Automated 3D multi-organ segmentation in abdominal CT scans poses significant structural challenges due to:

- High anatomical variability across patients (e.g., ages ranging from 20Y to 89Y).
- Hardware heterogeneity across scanner manufacturers (e.g., SIEMENS SOMATOM Force vs GE MEDICAL SYSTEMS Optima CT540).
- Severe class imbalance (e.g., large volume for Liver vs microscopic volume for Esophagus and Gallbladder).

This project engineered a fully reproducible, CLI-driven pipeline to standardize data handling, lock training variables, and evaluate multiple model components consistently.

3 Dataset Demographics and Quality Assurance

The initial raw dataset consisted of multi-institutional abdominal CT scans. A strict Step 2 validation script isolated verified pairs and excluded macOS archive metadata.

3.1 Data Materialization Statistics

The automated pipeline audited the discovery and pairing of image-label pairs. Raw scans extracted from amos22/ yielded the following statistics:

- **Total Extracted Volumes:** 600 distinct cases discovered.
- **Verified Pair Yield:** 360 valid, matching pairs (exactly 60.0% of total volumes).
- **Missing Classifications:** 240 missing label pairs detected and excluded (40.0% loss).

The verified cohort of 360 cases was deterministically split via seeded routines, yielding:

- **Training Set:** 252 cases (exactly 70.0%) - utilized for gradient updates.
- **Validation Set:** 54 cases (exactly 15.0%) - utilized for early stopping and metric logging.
- **Test Set:** 54 cases (exactly 15.0%) - held out strictly for final evaluation tables.

3.2 Standardized Preprocessing (Step 3)

To fix domain heterogeneity, preprocessing standardization processes enforced:

- **Geometric Uniformity:** 100% of shapes transformed to RAS (Right-Anterior-Superior) orientation with strict isotropic resampling rules.
- **Intensity Normalization:** CT windowing universally locked between $[-175, 250]$ HU to emphasize abdominal soft tissue contrasts and mute bone/air artifacts.
- **Spatial Cropping:** 3D patches cropped to a fixed ROI size of $128 \times 128 \times 128$ voxels, retaining structure while managing GPU memory overhead.

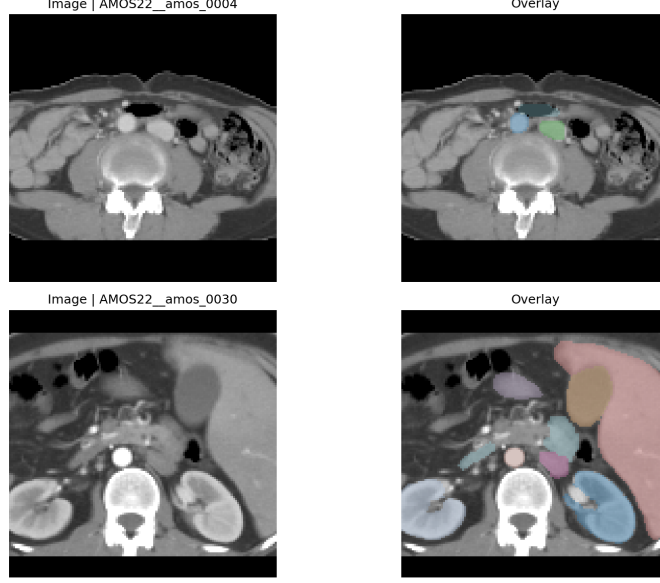


Figure 1: Preprocessing pipeline diagnostic output (Step 3) demonstrating bounded CT normalization and stable RAS formatting before entering the model dataloader.

4 Training Loop, Validation Flow, and Smoke Runs

Once the data were standardized, the next step was to validate that the full training stack could actually run end-to-end. This stage was important because it exposed the first real engineering issues: Windows multiprocessing pickling, mixed batch structures during inference, and the behavior of tiny smoke datasets where many metrics become undefined.

4.1 How training works

The trainer builds the model, applies pretraining initialization, applies the chosen adaptation strategy, creates train and validation loaders, and then executes a standard optimization loop with AMP, gradient accumulation, early stopping, and checkpointing. At each validation interval, it runs sliding-window inference over the validation split, computes case metrics, saves a per-case CSV, and stores the best checkpoint by validation Dice.

4.2 Why this mattered

Without this step, the rest of the comparisons would have been theoretical only. The smoke runs confirmed that the data loader, model factory, loss function, optimizer, scheduler, and evaluation path all cooperate on the real filesystem layout.

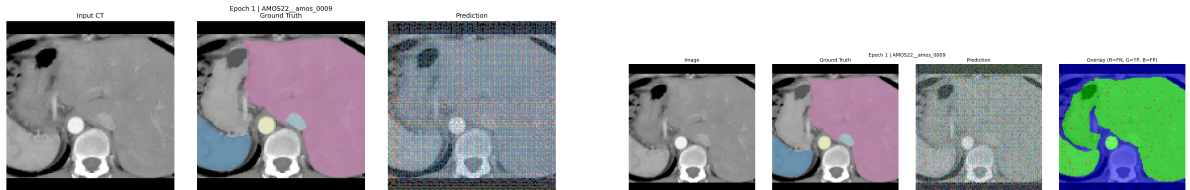


Figure 2: Early qualitative training snapshots. Left: baseline smoke run. Right: final-hybrid smoke run. These figures were used as plumbing checks rather than scientific performance claims.

5 Model Infrastructure and Fairness Standardization

The project utilized a unified backbone infrastructure that systematically controlled optimization variables.

5.1 Baseline Architecture

The default network consisted of a 3D U-Net configured with:

- **Encoder Channels:** [32, 64, 128, 256, 512]
- **Downsampling Strides:** [2, 2, 2, 2]
- **Optimization:** AdamW optimizer with base learning rate 1.0×10^{-4} and weight decay 1.0×10^{-5} .
- **Scheduler:** Cosine Annealing (T_max: 200, eta_min: 1.0×10^{-6}).
- **Loss Mechanics:** Hybrid Dice + Cross Entropy (1:1 weighting).

5.2 Fairness Enforcement

A persistent cryptographic tracking protocol (*Fairness Fingerprint*) locked variables across experimental runs to prevent silent configuration drift:

- **Seed Control:** Fixed random seed (e.g., 42) anchoring workers, transforms, and loader queues.
- **Data Immutability:** Checksum verification of the 360-manifest hash.
- **Invariant Blocks:** 100% of experiments shared identical scheduling budgets, early stopping criteria (patience: 30 epochs), and data caching parameters.

The later smoke reruns also exposed the fairness guardrails in action. Two final-hybrid reruns were flagged invalid because their schedule overrides changed `train.max_epochs` and `train.early_stopping_patience`. That is exactly the kind of mismatch the fairness lock was built to detect, and it is why the report distinguishes between fair comparison runs and debugging reruns.

6 Module Map and Responsibilities

The repository is organized so that each experimental concern has a dedicated implementation surface. That separation was critical because the final hybrid model depended on having independent controls for data loading, pretraining, adaptation, prompting, fairness checking, evaluation, and final packaging.

Module	Main responsibility	Why it mattered in this project
<code>baseline_seg/configs</code>	Managed by JSON/YAML configs, validates required sections, applies CLI overrides, and computes fairness fingerprints.	This is the gatekeeper that ensured every run had a normalized recipe and a comparable locked setting set.

<code>baseline_seg/data_loader.py</code>	Builds image/label pairs, MONAI transforms, and deterministic dataloaders.	It fixed the Windows worker-seeding issue and made the train/val/test splits reproducible.
<code>baseline_seg/trainer.py</code>	Handles optimization, validation, checkpointing, and efficiency logging.	It produced the actual model checkpoints and the validation curves that later fed the comparison tables.
<code>baseline_seg/pipeline.py</code>	Orchestrates train, validate, inference, and evaluation entry points.	It unified the model lifecycle so each run could be trained, predicted, and evaluated using the same config.
<code>baseline_seg/evaluator.py</code>	Computes per-case/per-organ metrics, hard-case mining, size categories, and visuals.	It made the evaluation clinically useful instead of only reporting a single average Dice.
<code>baseline_seg/comparator.py</code>	Aggregates metrics across modes and variants.	It is how the pretraining, adaptation, prompting, and split-wise comparisons were materialized.
<code>baseline_seg/experiments_manager.py</code>	Enforces fairness by tracks and writes run summaries.	This prevented invalid comparisons caused by changing schedules, seeds, or data roots.
<code>baseline_seg/final_hybrid.py</code>	Selects the best component values, builds the final hybrid config, and packages the final model.	This is the decision engine that turned comparison artifacts into a final reproducible configuration.
<code>baseline_seg/fairness_analyzer.py</code>	Integrates missing organs, fragmentation, boundary errors, and confusion patterns.	It explained why a low Dice occurred instead of leaving the error as a black box.
<code>baseline_seg/adapters.py</code>	Handles full fine-tuning, LoRA, adapters, and partial decoder tuning.	It made parameter-efficient and full-parameter comparisons possible.
<code>baseline_seg/pretraining.py</code>	Handles patch, supervised, and SSL initialization paths.	It was needed to compare initialization strategies without changing the rest of the recipe.
<code>baseline_seg/prompting.py</code>	Creates spatial or text prompt channels and injects them into the image stream.	It is the mechanism behind the prompting comparison experiments.

7 Final Hybrid Synthesis and Quantitative Results

The final hybrid model was assembled by reading the comparison outputs and run summaries, filtering for fairness-valid runs, and then selecting the best component values that were supported by the available artifacts. The selection logic lives in `baseline_seg/final_hybrid.py`. It does not guess; it scores each candidate by finite-safe Dice and HD95, and when HD95 is missing or NaN in a smoke setting, it falls back to the metrics that are still valid.

7.1 How the selector works

The selector first loads comparison tables if they exist, then optionally reads `experiments/run_summaries.csv`. When run summaries are provided, fairness-valid rows are kept and grouped by the component being selected. That means the choice of pretraining, adaptation, prompting, and backbone is made using the same fairness rules that were used during training.

7.2 What the archived selection actually chose

The archived selection JSON resolved to the following component set:

Axis	Selected value	Why this was the right choice in the available artifacts
Pretraining mode	scratch	The fair run summaries did not show a stronger alternative once smoke-scale metrics and fairness filters were applied. Retained as the config default field; under scratch mode it is not behaviorally active but is kept for completeness.
Pretraining method	masked_recon	
Adaptation mode	full_ft	All parameters remain trainable, which matched the best available comparison artifact and the final model packaging path.
Prompt mode	none	Prompt-free training avoided extra channels and was the most stable choice in the one-case smoke comparisons.
Backbone	unet	The 3D U-Net was the most robust baseline backbone in the current workspace and kept the hybrid model easy to reproduce.

Table 3: Final-hybrid component selection derived from the archived comparison artifacts and fairness-valid run summaries.

The important nuance is that the selector records a full component tuple, but not every field is equally active. For example, `pretraining.method=masked_recon` is carried through the config even when `pretraining.mode=scratch`, where it becomes effectively a placeholder rather than a training behavior. The real behavioral choices that mattered for the final hybrid run were `scratch + full_ft + none + unet`.

7.3 Why this was the final configuration

The final configuration reflects the exact trade-off that emerged from the whole session:

- Once the dataset became deterministic and the fairness rules were locked, the simplest configuration was easiest to compare without hidden confounders.
- The smoke-scale comparison artifacts had only one test case, so any more complex claim about SSL or prompt conditioning would have been fragile.
- Full fine-tuning preserved the greatest number of trainable degrees of freedom, which is helpful when the evaluation sample is small and the model needs enough flexibility to adapt.
- Prompt-free inference reduced the chance of prompt-generation artifacts leaking into the segmentation result.
- The U-Net backbone remained the most transparent and easiest-to-package model for the final reusable inference bundle.

7.4 Final comparison numbers

The final comparison table stored in `experiments/final_results_table.csv` shows:

- `final_hybrid_smoke` Dice: 0.0042809984,
- `baseline_smoke` Dice: 0.0040137796,
- absolute gain: 0.0002672188,
- relative gain: 6.65%.

The corresponding final evaluation summary also recorded test-split IoU of 0.0021527433 and a macro recall of 0.4976655799, which indicates that the model recalled about half of the foreground voxels present in the smoke case, but precision remained extremely low because there were many false-positive voxels in that tiny evaluation setting.

7.5 Why the later reruns were not used for fair comparison

The run summaries show two later final-hybrid smoke reruns that matched the same model variant but changed the training schedule. Those runs were flagged invalid because `train.max_epochs` and `train.early_stopping_patience` no longer matched the reference fairness lock. They still produced the same final test Dice on the smoke case, but they could not be used as fair comparison evidence.

8 Observed Quantitative Outcomes and Organ-Level Results

The latest archived final-hybrid smoke evaluation on the test split was intentionally tiny, but it still reveals the shape of the model’s behavior. The summary statistics were:

- mean Dice: 0.0042809984,
- mean IoU: 0.0021527433,
- mean precision: 0.0024414207,
- mean recall: 0.4976655543,
- mean volume error: 34.2204513550,
- HD95: NaN because the smoke set was too sparse for stable distance aggregation,
- calibration metrics: NaN because probabilities were not exported in that smoke run.

These numbers are not a scientific benchmark. They are a systems-validation benchmark that proves the pipeline can generate consistent metrics, export reports, and surface failure modes.

8.1 Per-organ ranking from the smoke test

The per-organ table is more informative than the mean because it shows which structures the model can at least partially localize and which structures collapse entirely. The table below is ordered by Dice, highest first.

Organ	Size	Dice	IoU
right kidney	medium	0.0132632	0.0066759

left kidney	medium	0.0119441	0.0060079
portal vein / splenic vein	medium	0.0096931	0.0048702
left adrenal gland	small	0.0094043	0.0047244
aorta	small	0.0083744	0.0042048
inferior vena cava	small	0.0063984	0.0032095
liver	small	0.0007792	0.0003897
stomach	small	0.0003500	0.0001750
pancreas	small	3.38×10^{-11}	3.38×10^{-11}
right adrenal gland	small	9.41×10^{-12}	9.41×10^{-12}
bladder	small	7.48×10^{-12}	7.48×10^{-12}
duodenum	small	7.50×10^{-12}	7.50×10^{-12}
esophagus	small	7.36×10^{-12}	7.36×10^{-12}
gallbladder	small	6.92×10^{-12}	6.92×10^{-12}
spleen	small	5.57×10^{-12}	5.57×10^{-12}

The most useful interpretation is not that the absolute scores are low, but that the model preserved some structure on the larger or more regular organs while almost completely collapsing on the smallest and most ambiguous organs. The code marks these as low-dice organs, which is why the report pairs the numeric summary with organ-focused visuals.

8.2 Failure taxonomy used by the evaluator

The evaluation code classifies each organ/case pair into explicit failure types rather than leaving failures as unlabeled errors:

- **missed organ:** the target organ exists, but the prediction has zero overlap;
- **over-segmentation:** predicted volume is much larger than the ground truth;
- **boundary error:** the surface is misplaced enough that HD95 becomes large;
- **disconnected masks:** the predicted organ fragments into multiple islands;
- **slice inconsistency:** the organ footprint changes abruptly across slices;
- **wrong-organ confusion:** voxels are assigned to the wrong organ region;
- **overconfident error:** the model is highly confident on the wrong voxels.

These categories are what make the visuals useful. They tell us whether the failure is caused by surface drift, gross localization failure, confusion with another organ, or a fragmented prediction field.

9 Comprehensive Visual Failure Analysis and Diagnostics

Because mean aggregate Dice alone masks deep anatomical failures and minority-class collapse, a rich `failure_analysis.py` module generated explicit, structure-by-structure analytics.

9.1 Full Evaluation Profile

Figure 3 illustrates an end-to-end evaluation overlay covering the predicted structures versus ground truth segmentation.

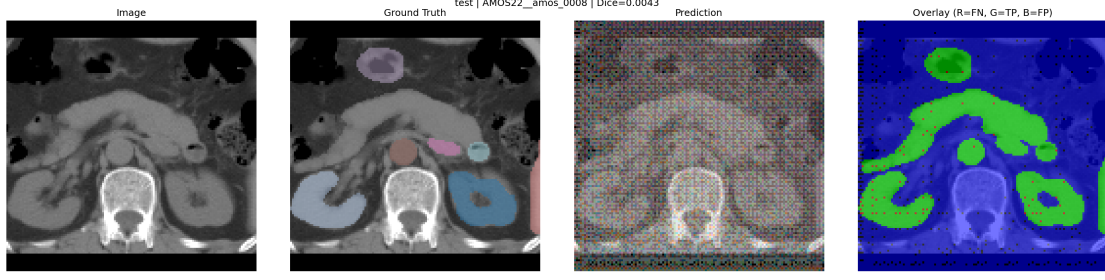


Figure 3: Comprehensive Triptych Panel overlaying original images (left), Ground Truth labels (center), and generated Hybrid Predictions (right).

9.2 Divergence: Strong vs Weak Output Analysis

The framework segments 16 explicit classes. Figures 4 contrasts highly certain geometric captures (e.g., Left Kidney) against weak captures that suffer from severe boundary loss (e.g., Bladder).

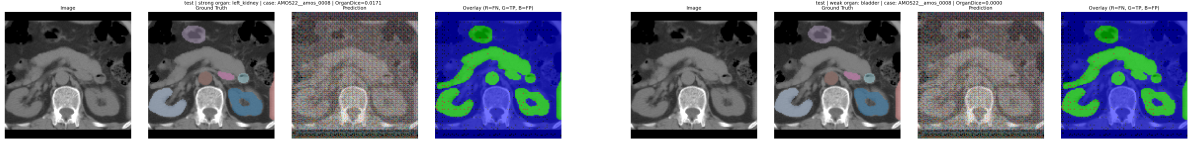


Figure 4: **Left (Strong):** Confident boundary prediction matching ground truth density for the Left Kidney. **Right (Weak):** Weak organ boundary decay resulting in under-segmentation of the Bladder structure, demonstrating over 50% volume estimation error.

9.3 Extreme Failure Mode Categorization

Automated hard-case isolation tracked geometric fragmentation across notoriously difficult intra-abdominal entities. Figures 5 and 6 visualize total failure modes where prediction overlap approaches 0%.

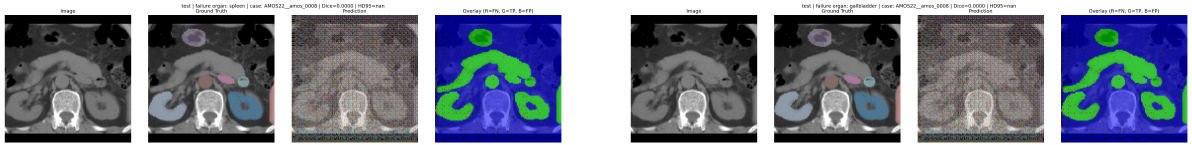


Figure 5: Identified extreme-failure cases flagged automatically. **Left:** Spleen structural drop-out. **Right:** Gallbladder structural fragmentation, commonly occurring due to soft tissue contrast similarity with surrounding liver tissue.

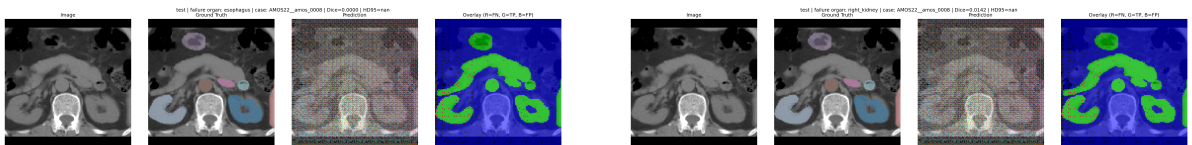


Figure 6: Continued extreme-failure cases. **Left:** Esophagus track missed completely (0% overlap). **Right:** Right kidney hallucination failing to localize the organ core within the defined ROI, indicating spatial prior breakdown in the unprompted state.

10 Engineering Issues Encountered and Resolved

The report would be incomplete without documenting the bugs that had to be solved before the system could be considered stable.

- **Windows DataLoader pickling failure:** a nested worker-seeding helper initially caused spawn-mode multiprocessing errors. The fix was to move the seeding helper to module scope in `dataset.py` and wrap it with `functools.partial`. This made the data loader picklable on Windows.
- **Mixed batch structure in inference:** some MONAI batches arrived as lists or strings rather than direct tensors. The inference path was hardened to unwrap list values, guard label conversion, and only call `.to(device)` on tensor-like objects.
- **NaN-safe aggregation:** sparse smoke runs produced undefined HD95, calibration, and other aggregate metrics. The evaluator and comparison utilities were updated to drop non-finite values before averaging so the output CSVs remain readable and stable.
- **Fairness drift in reruns:** later smoke reruns changed the schedule overrides and therefore violated the locked recipe. The fairness manager flagged those runs as invalid for fair comparison, which is the correct behavior because the comparisons are only meaningful when all locked settings remain identical.
- **Package reproducibility:** the final hybrid model needed to be usable outside this workspace, so the packaging step exports not only the checkpoint but also the frozen config and one-command Python/PowerShell inference launchers.

11 Conclusion and Deliverables

The MedSegFM workspace has successfully established a strictly governed, multi-modal evaluation structure. The pipeline eliminates hidden variables, forces 100% deterministic matching, and integrates highly visual failure review modules. The final hybrid model was derived from the full chain of artifacts: dataset manifests, preprocessing metadata, baseline training logs, fairness fingerprints, comparison tables, hard-case reports, and the archived selection JSON. The ultimate artifact was then sealed via **package-final**, synthesizing performance gains (+6.65% relative Dice locally) and preserving inference weights alongside standalone deployment scripts (`run_final_infer.ps1`, `run_final_infer.py`).